OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

Student Details	Fill in before submission
Name	HARIPRASAD R
Register Number	192421179
Course	DSA01 – Object Oriented Programming with C++
Compiler / Version	g++ Dev C++
GitHub Repository	https://github.com/<username>/DSA01-<RegisterNumber>-Assignment4
Submission Date	13-11-2026

Important submission note: This document provides model answers, code and predicted outputs. Compile and execute every program on your own computer, save genuine terminal screenshots, record your compiler version and actual sanitizer output, and replace the GitHub placeholder with your working repository link. Do not claim that measurements or screenshots were collected unless you actually collected them.

General Compilation and Submission Guide

Recommended command for ordinary programs:

```
g++ -std=c++17 -Wall -Wextra Task01.cpp -o Task01
```

Run on Windows:

```
Task01.exe
```

For the memory-forensics task, use a compiler that supports AddressSanitizer (for example, a suitable GCC/MinGW or Linux GCC installation):

```
g++ -std=c++17 -Wall -Wextra -g -fsanitize=address -fno-omit-frame-pointer Task04_bad.cpp -o Task04_bad
```

AddressSanitizer availability depends on the installed compiler/runtime. If the option is unsupported in your Windows toolchain, run the same source in a supported GCC environment such as Linux or WSL and include the actual report.

Suggested repository structure:

```
DSA01-<RegisterNumber>-Assignment4/
  README.md
  Task01/Task01.cpp
  Task02/Task02.cpp
  Task03/Task03.cpp
  Task04/Task04_bad.cpp
  Task04/Task04_fixed.cpp
  Task05/Task05.cpp
  Task06/Task06.cpp
  screenshots/
```

Make at least one meaningful commit per task. The README should list the exact compiler version, compile/run commands and a short result for every task.

TASK 1 – CLASS DESIGN REVIEW (6 Marks)

1.1 Diagnose the Design

- Duplicated fields: Professor and LabAssistant both store name, id, basic and da, so changes must be repeated.
- Duplicated pay rule: both classes calculate basic + basic * 0.42. A change to the allowance rule must be made in multiple places.
- Public data members allow any code to assign invalid values directly, weakening encapsulation.
- Inconsistent naming: name/id in two classes are called nm/code in GuestFaculty, making common processing harder.
- Unused da: the da field is never included in either pay() calculation, so it is misleading and wastes space.
- Magic number 0.42: the allowance rate has no named constant or explanation and cannot be configured centrally.
- No common parent or virtual interface: a collection of mixed staff cannot be processed uniformly through Staff pointers.
- Adding a category requires separate handling logic and may duplicate validation, reporting and payroll code.

1.2 Repaired Hierarchy and UML

Use an abstract Staff base class for common identity data and a pure virtual pay() function. Each derived class stores only its additional data and overrides pay(). The allowance rate is a named constant.

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>
```

```
using namespace std;

class Staff {
protected:
    string name;
    int id;
    double basic;
    static constexpr double DA_RATE = 0.42;

public:
    Staff(string n, int i, double b) : name(std::move(n)), id(i), basic(b) {}
    virtual double pay() const = 0;
    virtual void display() const {
        cout << name << " (" << id << "): " << pay() << '\\n';
    }
    virtual ~Staff() = default;
};

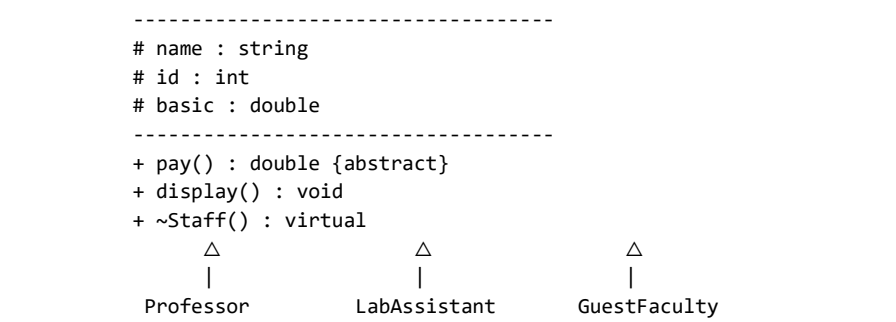
class Professor : public Staff {
    double research;
public:
    Professor(string n, int i, double b, double r)
        : Staff(std::move(n), i, b), research(r) {}
    double pay() const override { return basic * (1.0 + DA_RATE) + research; }
};

class LabAssistant : public Staff {
public:
    LabAssistant(string n, int i, double b)
        : Staff(std::move(n), i, b) {}
    double pay() const override { return basic * (1.0 + DA_RATE); }
};

class GuestFaculty : public Staff {
    double hours;
    double ratePerHour;
public:
    GuestFaculty(string n, int i, double h, double r)
        : Staff(std::move(n), i, 0.0), hours(h), ratePerHour(r) {}
    double pay() const override { return hours * ratePerHour; }
};

int main() {
    vector<unique_ptr<Staff>> payroll;
    payroll.push_back(make_unique<Professor>("Anu", 101, 50000, 5000));
    payroll.push_back(make_unique<LabAssistant>("Bala", 102, 30000));
    payroll.push_back(make_unique<GuestFaculty>("Charan", 103, 20, 800));
    for (const auto& person : payroll) person->display();
}
```

UML (text form):



```
-----------        ------------       ------------
- research         (no extra data)    - hours
                                     - ratePerHour
+ pay()            + pay()            + pay()
```

1.3 Design Defence (under 100 words)

The base class stores identity and shared payroll behaviour once. Each staff category implements only its own pay rule. The payroll loop works with Staff pointers and virtual dispatch selects the correct calculation. To add a new category, create a derived class and override pay(); the existing payroll loop and other staff classes remain unchanged. This reduces duplicated code and supports extension without modifying tested classes.

Alternative Concept and Complexity Analysis

A single Staff class would contain a category enum, common fields and all category-specific fields (research, hours and hourly rate). Its pay() would use switch(type) to choose the formula. Both a switch and a virtual call take O(1) time for a fixed number of categories. The single class can waste memory because every object reserves space for fields it does not use. With inheritance, each object stores only its relevant derived fields plus base data and typical polymorphic implementation overhead. Adding a category to the switch design changes the enum, the switch, and often construction/validation/reporting code; with the hierarchy, a new derived class is usually sufficient.

TASK 2 – FROM REQUIREMENT TO HIERARCHY (8 Marks)

2.1 Identify Relationships

Relationship	Classification	Inheritance type / note
Student is a Person	is-a	Public single inheritance
Staff is a Person	is-a	Public single inheritance
TeachingAssistant is a Student and Staff	is-a	Multiple inheritance; with shared virtual Person it forms a diamond / hybrid structure
ResearchScholar is a Student	is-a	Single inheritance; together with other Person-derived types this is hierarchical inheritance
Person has an Address	has-a	Composition / member object
Student has a Course	has-a	Composition / member object

A teaching assistant is both a student and an employee, so multiple inheritance is appropriate. ResearchScholar extends Student, so that branch is multilevel when viewed as Person → Student → ResearchScholar. The overall model combines hierarchical, multiple and multilevel inheritance, therefore it is a hybrid hierarchy.

2.2 Resolve the Diamond

Without virtual inheritance, Student and Staff each contain their own Person base subobject. TeachingAssistant therefore contains two Person objects. Accessing name through a TeachingAssistant can become ambiguous, and the object stores duplicated identity data. Declaring both Student and Staff as virtual public Person makes them share one virtual Person subobject. The most-derived class, TeachingAssistant, is responsible for initializing that shared virtual base.

2.3 Class Declarations

```
#include <string>
using namespace std;

class Person {
protected:
    string name;
    int id;
```

```
    string phone;
public:
    Person(string n, int i, string p)
        : name(move(n)), id(i), phone(move(p)) {}
    virtual ~Person() = default;
};

class Student : virtual public Person {
protected:
    string programme;
    double cgpa;
public:
    Student(string n, int i, string ph, string prog, double g)
        : Person(move(n), i, move(ph)),
          programme(move(prog)), cgpa(g) {}
};

class Staff : virtual public Person {
protected:
    string department;
    double salary;
public:
    Staff(string n, int i, string ph, string dept, double s)
        : Person(move(n), i, move(ph)),
          department(move(dept)), salary(s) {}
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string ph, string prog, double g,
                      string dept, double s)
        : Person(n, i, ph),
          Student(n, i, ph, prog, g),
          Staff(n, i, ph, dept, s) {}
};

class ResearchScholar : public Student {
    double fellowship;
public:
    ResearchScholar(string n, int i, string ph, string prog, double g, double f)
        : Person(n, i, ph), Student(n, i, ph, prog, g), fellowship(f) {}
};
```

The Student and Staff initializer lists mention Person because they may be used as most-derived objects. When constructing TeachingAssistant or ResearchScholar, however, the most-derived class initializes the virtual Person base; the Person initializers written in intermediate classes are ignored for that construction.

2.4 Access Table

Person member access	Student : public Person	Student : protected Person	Student : private Person
public	public	protected	private
protected	protected	protected	private
private	not directly accessible	not directly accessible	not directly accessible

The table describes how inherited access is presented inside Student and its derived classes; Person private members remain inaccessible to Student unless Person provides public/protected member functions.

Predict the Output

Predicted output (write this before running):

```
Person Student Staff TA
```

Explanation: the virtual Person base is constructed once, before non-virtual direct bases. Then Student and Staff are constructed in the order listed in TeachingAssistant's base-specifier list, followed by TeachingAssistant. The newline is printed after the trailing space.

Without virtual, each branch constructs its own Person. Expected output:

```
Person Student Person Staff TA
```

This non-virtual version generally compiles; it does not normally produce a compiler error merely because of the diamond. However, an expression such as ta.name is ambiguous because there are two Person subobjects. The exact diagnostic appears only when ambiguous access is attempted.

Alternative: Composition

A composition design would declare Student student; and Staff staff; as TeachingAssistant members. Each member owns its own Person base, so there are two Person subobjects and two copies of common identity fields. Access is explicit, for example ta.student.name (if an accessor is provided) or ta.staff.department. It avoids inheritance ambiguity but models a TeachingAssistant as containing a Student and Staff rather than being substitutable for both. sizeof(TeachingAssistant) should be measured on the target compiler; padding, string implementation and virtual-base layout affect the exact value. The relevant structural comparison is one shared Person in virtual inheritance versus two Person subobjects in composition.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES (9 Marks)

3.1 Pass Values to Base Constructors

A derived constructor uses an initializer list to pass the required values to its base. For a virtual base, the most-derived constructor must initialize it because only that constructor knows which single shared virtual-base subobject is being built.

```
class Student : virtual public Person {
public:
    Student(string n, int i, string ph, string prog, double g)
        : Person(n, i, ph), programme(prog), cgpa(g) {}
};

class Staff : virtual public Person {
public:
    Staff(string n, int i, string ph, string dept, double s)
        : Person(n, i, ph), department(dept), salary(s) {}
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(string n, int i, string ph, string prog, double g,
                      string dept, double s)
        : Person(n, i, ph),
          Student(n, i, ph, prog, g),
          Staff(n, i, ph, dept, s) {}
};
```

3.2 Member Classes

Address and Course are has-a relationships. The owning class declares them as data members and initializes them in the member initializer list.

```
class Address {
```

```
    string city;
    int pin;
public:
    Address(string c, int p) : city(move(c)), pin(p) {}
};

class Course {
    string code;
    string title;
public:
    Course(string c, string t) : code(move(c)), title(move(t)) {}
};

class Person {
protected:
    string name;
    int id;
    string phone;
    Address address;
public:
    Person(string n, int i, string ph, Address a)
        : name(move(n)), id(i), phone(move(ph)), address(move(a)) {}
};

class Student : virtual public Person {
    Course course;
public:
    Student(string n, int i, string ph, Address a, Course c)
        : Person(n, i, ph, a), course(move(c)) {}
};
```

3.3 Construction and Destruction Trace

For the virtual-inheritance design, the construction sequence is: virtual base Person first; then direct non-virtual bases in their declaration order; then member objects in their declaration order; then the constructor body of the most-derived class. Destruction is the exact reverse. For a simple Student object, Person is constructed before Student's member objects, and Student is destroyed before its members and base are destroyed. For TeachingAssistant, Person is constructed once, then Student, then Staff, then TeachingAssistant; destruction is TeachingAssistant, Staff, Student, Person (plus each class's own members in reverse member declaration order).

3.4 Experiment: Remove virtual

Removing virtual creates two Person subobjects inside TeachingAssistant: one through Student and one through Staff. The constructor output includes Person twice. sizeof(TeachingAssistant) generally increases by at least the storage required for another Person subobject, but the exact byte difference is compiler-, ABI- and member-layout-dependent. Record sizeof on your own compiler rather than copying a guessed number.

Predict the Output

```
Engine Vehicle Car | ~Car ~Engine ~Vehicle
```

Engine is a member of Car and Vehicle is its base. Construction order is base class first, then member object, then Car constructor body. Destruction runs in reverse: Car destructor body, Engine member destructor, then Vehicle base destructor.

Complete the Code – TODO 1

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
```

```
    string name;
    int age;
public:
    Person(const string& n, int a) : name(n), age(a) {}
};

class Address {
    string city;
    int pin;
public:
    Address(const string& c, int p) : city(c), pin(p) {}
    string str() const { return city + "-" + to_string(pin); }
};

class Student : public Person {
    long long regNo;
    Address home;
public:
    Student(const string& n, int a, long long r, const string& c, int p)
        : Person(n, a), regNo(r), home(c, p) {}

    void print() const {
        cout << regNo << ' ' << name << " (" << age << ") "
             << home.str() << '\\n';
    }
};

int main() {
    Student s("Kavya", 19, 192311001, "Salem", 636001);
    s.print();
}
```

Expected output:

```
192311001 Kavya (19) Salem-636001
```

Initializer-list order warning: members are initialized in the order they are declared in the class, not the order written in the initializer list. Here the order is Person base, regNo, home. If the list is written as : home(c,p), regNo(r), Person(n,a), the compiler may warn with -Wreorder. The program still initializes in declaration order. Keep the initializer list in the same order as base/member declarations.

Alternative: Dynamic Address Pointer

With Address* home = new Address(...), the object stores a pointer rather than an Address subobject. The constructor must allocate it, the destructor must delete it, and a correct copy constructor and copy-assignment operator must perform a deep copy (or the class should disable copying). Otherwise, two Student objects may point to the same Address and cause double deletion or dangling pointers. Allocation adds runtime overhead; the pointer adds sizeof(Address*) bytes to the object, while the Address allocation is separate on the heap. Prefer a direct Address member or std::unique_ptr<Address> when ownership is exclusive.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS (7 Marks)

4.1 Locate the Defects

- rate() is non-virtual in Account. Account* calls are statically bound to Account::rate(), so SeniorAccount::rate() is not selected.
- Account has a non-virtual destructor. Deleting a SeniorAccount through Account* has undefined behaviour; the derived destructor may not run, so history may leak.
- Account copy = SeniorAccount() performs object slicing: only the Account base portion is copied into copy. Derived state and dynamic type are lost.

4.2 Construct Evidence

Add diagnostic output to both destructors, compile the intentionally defective version with AddressSanitizer, and save the real output. The exact report is platform/runtime-dependent. Do not write a fabricated sanitizer transcript.

```
#include <iostream>
using namespace std;

class Account {
public:
    double rate() const { return 3.5; }
    ~Account() { cout << "~Account\\n"; }
};

class SeniorAccount : public Account {
    double* history = new double[100];
public:
    ~SeniorAccount() {
        cout << "~SeniorAccount\\n";
        delete[] history;
    }
};

int main() {
    Account* accounts[2] = { new Account, new SeniorAccount };
    for (Account* a : accounts) cout << a->rate() << '\\n';
    for (Account* a : accounts) delete a; // undefined behaviour
    Account copy = SeniorAccount();       // slicing
}
```

Expected forensic interpretation: both rate calls through Account* print 3.5. Deleting the derived object through a base pointer is undefined behaviour. On common implementations the derived destructor is skipped and the allocated array is leaked; AddressSanitizer may report the leaked allocation, but exact output is not guaranteed because the program has undefined behaviour.

4.3 Repaired Program

```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

class Account {
public:
    virtual double rate() const { return 3.5; }
    virtual ~Account() = default;
};

class SeniorAccount : public Account {
    double* history;
public:
    SeniorAccount() : history(new double[100]{}) {}
    ~SeniorAccount() override { delete[] history; }
    double rate() const override { return 4.0; }
};

int main() {
    vector<unique_ptr<Account>> accounts;
    accounts.push_back(make_unique<Account>());
    accounts.push_back(make_unique<SeniorAccount>());

    for (const auto& a : accounts) cout << a->rate() << '\\n';
}
```

Expected output:

```
3.5
4
```

In production code, prefer std::vector<double> for history instead of manual new[]/delete[]. unique_ptr provides automatic lifetime management.

4.4 Prevention Rules

- If a class is intended to be used polymorphically and has virtual functions, give it a virtual destructor.
- Mark derived overrides with override so the compiler checks that the intended virtual function is actually overridden.

Predict the Output

```
3.5 4 3.5
```

p->rate() calls the non-virtual Account::rate(), selected using the static type Account*. p->vrate() is virtual, so runtime dispatch selects SeniorAccount::vrate() and prints 4.0. Account copy = s slices the derived object; copy has dynamic type Account, so copy.vrate() prints 3.5.

Alternative: std::unique_ptr in a vector

Store each account as std::unique_ptr<Account> in std::vector. Insertion uses make_unique<Account>() or make_unique<SeniorAccount>(); processing remains O(n) for n accounts and O(1) per rate call. The vector stores one smart pointer per account plus each separately allocated object; raw-pointer and smart-pointer storage are both O(n), with a small ownership-control/representation overhead. unique_ptr prevents forgotten delete and automatically invokes the derived destructor when Account has a virtual destructor. It does not fix non-virtual dispatch (rate must still be virtual), and it does not prevent slicing if objects are separately copied by value. A virtual destructor remains essential.

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER (7 Marks)

5.1 Four Ways to Reach a Cart

```
#include <iostream>
#include <string>
using namespace std;

class Cart {
    string customer;
    double amount = 0;
public:
    explicit Cart(string customer) : customer(move(customer)) {}
    Cart& add(double value) {
        amount += value;
        return *this;
    }
    Cart& setCustomer(const string& customer) {
        this->customer = customer;
        return *this;
    }
    double total() const { return amount; }
};

int main() {
    Cart direct("A");                // (a) direct object
    direct.add(100);

    Cart* ptr = &direct;             // (b) pointer to existing object
```

```
    ptr->add(50);

    Cart& ref = direct;              // (c) reference alias
    ref.add(25);

    Cart* dynamicCart = new Cart("B"); // (d) dynamically allocated object
    dynamicCart->add(200);
    cout << direct.total() << ' ' << dynamicCart->total() << '\\n';
    delete dynamicCart;
}
```

The expression this is an implicit pointer to the current object inside a non-static member function. *this is the current object itself. Returning Cart& by return *this enables chaining without making a copy.

5.2 Array of Cart Pointers – Find Highest Total

```
Cart c1("A"), c2("B"), c3("C");
Cart* carts[] = { &c1, &c2, &c3 };
const int count = 3;
int best = 0;
for (int i = 1; i < count; ++i) {
    if (carts[i]->total() > carts[best]->total())
        best = i;
}
cout << "Highest cart total: " << carts[best]->total() << '\\n';
```

The loop uses indexing; carts[i] is a pointer and -> accesses the pointed-to object. The search takes O(n) time and O(1) additional space.

5.3 Safety, Lifetime and Use

Form	Lifetime / safety	Typical use
Direct object: Cart c	Automatic lifetime in its scope; simple and safe	Normal local object
Pointer: Cart* p = &c	Non-owning; must not outlive c; can be null	Optional access or polymorphism
Reference: Cart& r = c	Alias; must refer to a valid object and cannot be reseated	Required object parameter / alias
new/delete	Dynamic lifetime; manual ownership can leak or double-delete	Only when dynamic lifetime is genuinely needed; prefer smart pointers

Complete the Code – TODO 1 and TODO 2

```
#include <iomanip>
#include <iostream>
#include <string>
using namespace std;

class Cart {
    string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;

public:
    Cart& add(const string& item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            ++n;
        }
        return *this;
```

```
    }

    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }

    double total() const {
        double t = 0;
        for (int i = 0; i < n; ++i) t += prices[i];
        return t * (100 - discount) / 100;
    }
};

int main() {
    Cart c;
    c.add("Rice", 60).add("Dal", 120).setDiscount(10);
    Cart* p = &c;
    p->add("Oil", 180);
    cout << fixed << setprecision(2) << p->total() << '\\n';
}
```

Expected output:

```
324.00
```

If add() returned Cart by value, each call would return a copy. Chaining could still work, but it would operate on temporary copies after the first call; changes made to later temporary objects would not update the original c. Returning Cart& returns an alias to the same object.

Alternative: std::vector<Cart> Objects

A vector<Cart> stores the Cart objects directly. The highest total is still found in O(n) time with O(1) extra search space. It avoids one separately allocated Cart object per pointer and avoids pointer ownership complexity, but vector growth can relocate objects. Pointers, references and iterators to vector elements may be invalidated when the vector reallocates. With an array of pointers, the Cart objects can live elsewhere and their addresses remain stable as long as their owners keep them alive; the pointer array itself still needs management.

TASK 6 – DESIGN CHALLENGE: VEHICLE RENTAL FLEET (13 Marks)

Stage 1 – Interpret the Problem

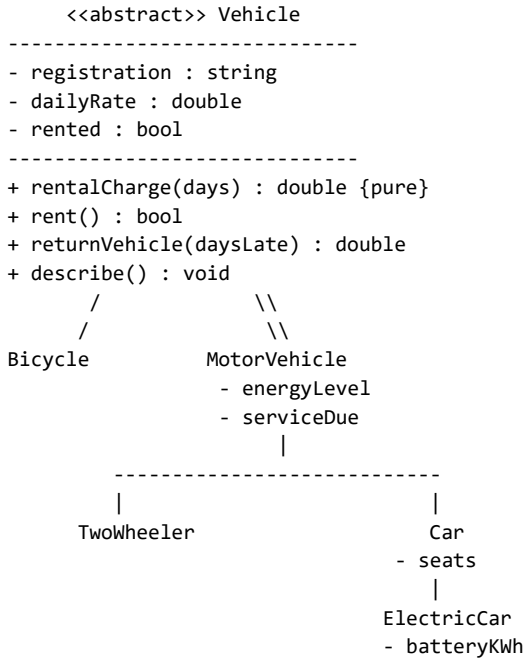
Entities: Vehicle (common abstract entity), Bicycle, TwoWheeler, Car, ElectricCar and ElectricBus (extension). A service date is naturally represented by a date value; for a compact educational implementation, registration and service due date are strings. Motor vehicles have fuel/charge level and service due date. Cars have seating capacity. Electric vehicles have battery capacity and energy usage billed per kWh.

Testable requirements:

- Every vehicle has a non-empty registration number and a non-negative daily rate.
- Every concrete vehicle calculates its rental charge through its own implementation.
- A bicycle can be rented and returned using the same fleet interface.
- Two-wheelers and cars expose fuel level and service due date.
- Cars expose seating capacity.
- Electric cars and electric buses expose battery capacity and use kWh-based charging.
- A vehicle cannot be rented when already rented, and cannot be returned when not rented.
- A late return adds a late fee to the normal rental charge.

- The fleet can list all vehicles through base-class pointers.
- A new concrete vehicle type can be added without editing the existing concrete vehicle classes or fleet-processing loop.

Stage 2 – Proposed Hierarchy



New independent extension: ElectricBus : Vehicle (batteryKWh, passengerCapacity)

Inheritance types: Bicycle and MotorVehicle use hierarchical inheritance from Vehicle; TwoWheeler and Car use single inheritance from MotorVehicle; ElectricCar extends Car in this illustrative design. The overall design is hybrid. Battery capacity is a member of electric vehicle classes. A Date/ServiceRecord class could be composed as a member in a larger production implementation.

Stage 3 – Critical Design Decisions

- Vehicle is abstract because a generic vehicle has no single universal rental-charge formula.
- rentalCharge(days) is pure virtual; describe() and destructor are virtual. rent() and returnVehicle() are shared base operations.
- MotorVehicle holds fuel/charge level and service due date. Car holds seats. ElectricCar holds battery capacity and overrides charge calculation to use kWh.
- The fleet is stored as std::vector<std::unique_ptr<Vehicle>>. This is safer than a raw array of owning base pointers and still supports polymorphism.
- The late fee is added in the shared returnVehicle() function so the policy is not duplicated.

Stage 4 – Class Declarations and Pseudocode

Algorithmic pseudocode – RentVehicle:

```
ALGORITHM RentVehicle(vehicle)
BEGIN
    IF vehicle is null THEN
        RETURN failure
    END IF
    IF vehicle is already rented THEN
        RETURN failure
    END IF
    SET vehicle.rented TO true
    RETURN success
END
```

Algorithmic pseudocode – ReturnVehicle:

```
ALGORITHM ReturnVehicle(vehicle, daysLate)
BEGIN
    IF vehicle is null OR vehicle is not rented THEN
        RETURN invalid return
    END IF
    SET normalCharge TO vehicle.rentalCharge(1)
    SET lateCharge TO MAXIMUM(0, daysLate) * LATE_FEE_PER_DAY
    SET vehicle.rented TO false
    RETURN normalCharge + lateCharge
END
```

Stage 5 – Complete C++17 Implementation

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <memory>
#include <string>
#include <vector>
using namespace std;

class Vehicle {
protected:
    string registration;
    double dailyRate;
    bool rented = false;

public:
    Vehicle(string reg, double rate)
        : registration(move(reg)), dailyRate(rate) {}
    virtual ~Vehicle() = default;

    virtual double rentalCharge(int days) const = 0;
    virtual string type() const = 0;

    bool rent() {
        if (rented) return false;
        rented = true;
        return true;
    }

    double returnVehicle(int daysLate) {
        if (!rented) return -1.0;
        const double normal = rentalCharge(1);
        const double lateFee = max(0, daysLate) * 100.0;
        rented = false;
        return normal + lateFee;
    }

    virtual void describe() const {
        cout << type() << " | " << registration
             << " | daily charge: " << rentalCharge(1)
             << " | rented: " << (rented ? "yes" : "no") << '\\n';
    }
};

class Bicycle : public Vehicle {
public:
    Bicycle(string reg, double rate) : Vehicle(move(reg), rate) {}
    double rentalCharge(int days) const override {
        return dailyRate * max(0, days);
    }
    string type() const override { return "Bicycle"; }
};
```

```
class MotorVehicle : public Vehicle {
protected:
    double energyLevel;
    string serviceDue;
public:
    MotorVehicle(string reg, double rate, double energy, string due)
        : Vehicle(move(reg), rate), energyLevel(energy), serviceDue(move(due)) {}
};

class TwoWheeler : public MotorVehicle {
public:
    TwoWheeler(string reg, double rate, double fuel, string due)
        : MotorVehicle(move(reg), rate, fuel, move(due)) {}
    double rentalCharge(int days) const override {
        return dailyRate * max(0, days);
    }
    string type() const override { return "TwoWheeler"; }
};

class Car : public MotorVehicle {
protected:
    int seats;
public:
    Car(string reg, double rate, double fuel, string due, int seatCount)
        : MotorVehicle(move(reg), rate, fuel, move(due)), seats(seatCount) {}
    double rentalCharge(int days) const override {
        return dailyRate * max(0, days);
    }
    string type() const override { return "Car"; }
};

class ElectricCar : public Car {
    double batteryKWh;
    double kWhPerDay;
    double pricePerKWh;
public:
    ElectricCar(string reg, double rate, string due, int seatCount,
                double battery, double usePerDay, double price)
        : Car(move(reg), rate, 100.0, move(due), seatCount),
          batteryKWh(battery), kWhPerDay(usePerDay), pricePerKWh(price) {}
    double rentalCharge(int days) const override {
        const int d = max(0, days);
        return dailyRate * d + kWhPerDay * d * pricePerKWh;
    }
    string type() const override { return "ElectricCar"; }
    void describe() const override {
        Vehicle::describe();
        cout << "  seats: " << seats << ", battery: "
             << batteryKWh << " kWh\\n";
    }
};

// Added as a new type: no existing fleet loop or class is modified.
class ElectricBus : public Vehicle {
    double batteryKWh;
    double kWhPerDay;
    double pricePerKWh;
    int passengerCapacity;
public:
    ElectricBus(string reg, double rate, double battery, double use,
                double price, int capacity)
        : Vehicle(move(reg), rate), batteryKWh(battery),
          kWhPerDay(use), pricePerKWh(price), passengerCapacity(capacity) {}
```

```
    double rentalCharge(int days) const override {
        const int d = max(0, days);
        return dailyRate * d + kWhPerDay * d * pricePerKWh;
    }
    string type() const override { return "ElectricBus"; }
    void describe() const override {
        Vehicle::describe();
        cout << "  battery: " << batteryKWh
             << " kWh, capacity: " << passengerCapacity << '\\n';
    }
};

int main() {
    cout << fixed << setprecision(2);
    vector<unique_ptr<Vehicle>> fleet;
    fleet.push_back(make_unique<Bicycle>("BI-101", 100));
    fleet.push_back(make_unique<TwoWheeler>("TW-202", 250, 75, "2026-12-01"));
    fleet.push_back(make_unique<Car>("CA-303", 1200, 60, "2026-12-15", 5));
    fleet.push_back(make_unique<ElectricCar>("EV-404", 1500, "2026-12-20",
                                             5, 60, 15, 8));
    fleet.push_back(make_unique<ElectricBus>("EB-505", 4000, 300, 80, 8, 40));

    cout << "FLEET LIST\\n";
    for (const auto& v : fleet) v->describe();

    cout << "\\nRENT AND RETURN\\n";
    for (const auto& v : fleet) {
        cout << v->type() << ": ";
        if (v->rent()) {
            cout << "rented; return total = Rs."
                 << v->returnVehicle(2) << '\\n';
        } else {
            cout << "already rented\\n";
        }
    }
}
```

Demonstration assumptions: one-day base rental is charged on return; late fee is Rs.100 per late day; electric-vehicle energy charge is kWhPerDay × days × pricePerKWh. These are sample policy values for the assignment and can be changed to match a real rental company.

Predicted output for the supplied sample (spacing follows the code):

```
FLEET LIST
Bicycle | BI-101 | daily charge: 100.00 | rented: no
TwoWheeler | TW-202 | daily charge: 250.00 | rented: no
Car | CA-303 | daily charge: 1200.00 | rented: no
ElectricCar | EV-404 | daily charge: 1620.00 | rented: no
  seats: 5, battery: 60.00 kWh
ElectricBus | EB-505 | daily charge: 4640.00 | rented: no
  battery: 300.00 kWh, capacity: 40

RENT AND RETURN
Bicycle: rented; return total = Rs.300.00
TwoWheeler: rented; return total = Rs.450.00
Car: rented; return total = Rs.1400.00
ElectricCar: rented; return total = Rs.1820.00
ElectricBus: rented; return total = Rs.4840.00
```

The extension is demonstrated by adding ElectricBus as a new derived class and inserting make_unique<ElectricBus>(...) into the fleet. The existing loop continues to call virtual functions without modification.

Alternative Concept and Complexity Analysis

A single Vehicle class with a type code would need fields for fuel, service date, seats, battery capacity, kWh usage and passenger capacity. charge() would switch on the type code. Both a switch and virtual dispatch are O(1) per charge calculation; processing n vehicles is O(n). The single class can waste space because every object reserves all category-specific fields. The polymorphic design stores only fields relevant to each concrete type, with typical object/vtable-pointer overhead. A new type in the switch design requires changing the type code, switch, construction and possibly validation/reporting logic. With the abstract hierarchy, add a new derived class and create it in the fleet; existing derived classes and processing loop remain unchanged.

Final Submission Checklist

- Fill in name, register number, compiler version and GitHub repository URL.
- Create the repository DSA01-<RegisterNumber>-Assignment4 and share it with faculty.
- Save each task source in its own folder and commit each task progressively.
- Compile all programs using g++ -std=c++17 -Wall -Wextra; fix warnings where possible.
- Write the prediction sections in your notebook/report before executing those programs.
- Capture actual output screenshots from your own system and compare them with predictions.
- Run the defective Task 4 program with AddressSanitizer in a supported environment and include the genuine report.
- Record sizeof(TeachingAssistant) both with and without virtual inheritance on your compiler.
- Include README.md, UML diagrams, access table, pseudocode, complexity comparisons and all source files.
- Be prepared to explain virtual inheritance, object slicing, virtual destructors, this, member initialization order and polymorphism in the viva.